Logiciel de gestion pour quincaillerie

Recommandations : base de données et architecture

Ce document rassemble l'analyse des cas de figure pour installer le logiciel Windows (Flutter, fichier .exe) dans une quincaillerie afin de suivre ses activités. Il sert de base pour configurer ensuite chaque cas.

1. Principe : un logiciel .exe et sa base de données

Un logiciel .exe ne contient pas la base de données dans son code. Il lit et écrit des données stockées ailleurs. Il existe deux grandes façons de faire.

Architecture 1 : base locale (sur le PC)

```
[ Logiciel .exe ]  <-->  [ Fichier de base de données sur le même PC ]
```

- Les données sont dans un fichier (par exemple provenderie.db) sur l'ordinateur.
- Aucun serveur ni internet nécessaire ; pratique pour un seul poste.
- Si plusieurs ordinateurs utilisent le logiciel, chacun a ses propres données, non partagées.

Architecture 2 : base centrale (serveur + API)

```
[ .exe PC 1 ] --+
[ .exe PC 2 ] --+--> [ API ] <--> [ Base de données serveur ]
[ .exe PC 3 ] --+
```

- La base est sur un serveur ; tous les postes voient les mêmes données en temps réel.
- Le logiciel envoie des requêtes HTTP à une API, et c'est l'API qui parle à la base.
- Il faut une connexion réseau ou internet.

2. Types de bases de données existants

Bases relationnelles (SQL) : tables, lignes, colonnes

Base	Usage typique
SQLite	Locale : un simple fichier, sans serveur à installer
MySQL / MariaDB	Serveur : très répandu, courant avec Symfony
PostgreSQL	Serveur : puissant et robuste
SQL Server	Serveur : écosystème Microsoft

Elles sont idéales pour la gestion (produits, stocks, achats, factures) : les données sont structurées et reliées entre elles.

Bases NoSQL : documents ou clés-valeurs

Base	Usage typique
Firebase / Firestore	Cloud de Google, synchronisation en temps réel
MongoDB	Serveur, données en documents JSON
Hive / Isar	Locales, rapides, spécifiques à Flutter

Elles sont plus souples, mais moins adaptées aux données de gestion très liées entre elles (stocks, comptabilité).

Stockage simple

shared_preferences garde de petits réglages (dernier utilisateur, thème, etc.). Ce n'est pas une vraie base de données.

Outils Flutter correspondants

Besoin	Outil Flutter
Base locale SQL sur Windows	drift ou sqflite_common_ffi (basés sur SQLite)
Parler à une API	http ou dio
Petits réglages	shared_preferences

3. Ce que le logiciel devra gérer dans la quincaillerie

Activité	Exemples de données
Articles	Référence, désignation, catégorie, prix d'achat et de vente, stock minimum
Ventes / caisse	Ticket, lignes de vente, remises, mode de paiement
Stock	Entrées, sorties, inventaire, alertes de rupture
Achats	Fournisseurs, bons de commande, réceptions
Clients	Fiche client, ventes à crédit, paiements partiels
Documents	Factures, devis, bons de livraison
Suivi	Caisse du jour, bénéfices, articles les plus vendus

Ces données sont très liées entre elles (une vente touche le stock, la caisse et le client) : une base relationnelle SQL est donc le bon choix.

4. Les cas de figure

Cas 1 : un seul ordinateur (une caisse)

```
[ .exe ] <--> [ fichier SQLite sur le PC ]
```

Avantages

- Très simple, aucun serveur, fonctionne sans internet, rapide.

Inconvénients

- Les données ne sont que sur ce PC : sauvegardes indispensables.

Outils : drift ou sqflite_common_ffi.

Cas 2 : plusieurs ordinateurs dans le même magasin

```
[ .exe caisse ] --+
[ .exe bureau ] --+--> [ PC serveur du magasin :
[ .exe stock  ] --+      base MySQL / PostgreSQL ]
                         (réseau local, sans internet)
```

Avantages

- Tous voient le même stock en temps réel.
- Fonctionne sans internet (réseau local).

Inconvénients

- Il faut installer et entretenir une base sur un PC serveur qui reste allumé.

Outils : MySQL ou PostgreSQL sur le PC serveur, accès via une API locale.

Cas 3 : plusieurs magasins ou accès à distance par le patron

```
[ .exe magasin 1 ]  --+
[ .exe magasin 2 ]  --+--> [ API ] <--> [ Base sur serveur internet ]
[ Patron à distance ] -+
```

Avantages

- Données centralisées, consultation de n'importe où.

Inconvénients

- Il faut internet et un serveur hébergé, plus un backend (API) à développer.

Outils : API (Symfony ou autre), MySQL ou PostgreSQL hébergé, package http ou dio.

Cas 4 : le magasin doit vendre même sans connexion

```
[ .exe ] <--> [ SQLite locale ] <==> synchronisation <==> [ Serveur ]
```

Avantages

- La vente n'est jamais bloquée par une coupure de connexion.

Inconvénients

- Le plus complexe : il faut gérer les conflits de données (deux postes qui modifient le même article).

Outils : SQLite locale + API + mécanisme de synchronisation.

5. Recommandation

Situation	Choix conseillé
1 caisse, 1 magasin	**Cas 1 : SQLite locale**
2 à 5 postes dans le même magasin	**Cas 2 : serveur local**
Plusieurs magasins	**Cas 3**, ou **Cas 4** si internet n'est pas fiable

Pour démarrer, le cas 1 est conseillé : c'est le plus rapide à construire et le plus fiable pour une première version à installer chez un client. Si le magasin grandit, on pourra passer au cas 2 ou 3 sans refaire l'interface, à condition de bien structurer le code dès le départ.

6. Bonnes pratiques valables dans tous les cas

- **Sauvegardes automatiques :** une copie quotidienne de la base sur une clé USB ou un autre disque. Sans cela, une panne du PC fait perdre toutes les données du commerçant.
- **Coupures de courant :** SQLite gère bien les arrêts brutaux, mais un onduleur reste recommandé.
- **Mots de passe :** stockés chiffrés (hachés), jamais en clair.
- **Rôles :** par exemple, un vendeur ne doit pas pouvoir modifier les prix ni supprimer une vente.

7. À éviter : connecter le .exe directement à MySQL

Techniquement possible, mais déconseillé :

- Les identifiants de la base seraient écrits dans le logiciel, et n'importe qui peut les récupérer en analysant le .exe.
- Il faudrait ouvrir la base sur internet, ce qui est risqué.
- Aucune règle métier ne serait vérifiée côté serveur.

La bonne pratique est de passer par une **API** qui vérifie la connexion et les droits de l'utilisateur, puis lit ou écrit dans la base.

8. Application Symfony existante

L'application Symfony actuelle est conçue pour l'élevage et la provenderie (bandes, aliments, vaccinations). Pour la quincaillerie, les tables sont différentes : elle ne peut pas être réutilisée telle quelle. Selon le cas choisi, on construit soit une base locale directement dans Flutter, soit un nouveau backend.

9. Prochaine étape : configurer les cas de figure

Pour chaque cas, la configuration portera sur :

- **Cas 1 :** ajouter SQLite à Flutter, créer les tables (articles, ventes, clients...), brancher la page de connexion et les sauvegardes.
- **Cas 2 :** installer la base sur le PC serveur, créer l'API locale, connecter les postes en réseau local.

- **Cas 3 :** héberger la base et l'API, sécuriser les accès, connecter les magasins.
- **Cas 4 :** ajouter la base locale, puis la synchronisation avec le serveur.